UNIVERSIDADE FEDERAL DE ITAJUBÁ

SISTEMAS DE INFORMAÇÃO

RELATORIO DO MIDDLEWARE: APACHE PULSAR

2021017110 - João Leão Guedes Filho
2023010066 - Pedro Venâncio dos Santos
2023003929 - Breno Vieira Nogueira Carneiro
2023013569 - Rodrigo Braz de Oliveira
2023008237 - Wesley Batista Luz

XRSC09 - SISTEMAS DISTRIBUÍDOS

Prof. Rafael Frinhani

INSTITUTO DE MATEMÁTICA E COMPUTAÇÃO
UNIFEI - Itajubá

2026

UNIVERSIDADE FEDERAL DE ITAJUBÁ
INSTITUTO DE MATEMÁTICA E COMPUTAÇÃO
SISTEMAS DE INFORMAÇÃO
XRSC09 - SISTEMAS DISTRIBUÍDOS

Relatorio do Middleware: Apache Pulsar

1 Introdução

Este relatório apresenta um estudo sobre o Apache Pulsar, middleware de comunicação distribuída utilizado em cenários de mensageria e processamento de eventos em tempo real. O trabalho foi desenvolvido no contexto da disciplina de Sistemas Distribuídos, com o objetivo de compreender os fundamentos da tecnologia, seu paradigma de comunicação, sua arquitetura e suas possibilidades de aplicação em sistemas modernos.

A escolha do Apache Pulsar se justifica por sua proposta de oferecer uma infraestrutura escalável e confiável para troca de mensagens entre processos distribuídos. Por meio do modelo *Publish-Subscribe* (Pub/Sub), a tecnologia permite que produtores e consumidores se comuniquem de forma indireta, utilizando tópicos e assinaturas. Essa abordagem reduz o acoplamento entre os componentes do sistema e favorece a construção de aplicações mais flexíveis, tolerantes a falhas e preparadas para crescimento.

Além da análise teórica, o trabalho contempla o desenvolvimento de um protótipo funcional, implementado em Python e executado com apoio do Docker. O protótipo demonstra a comunicação entre um cliente e um servidor utilizando o Apache Pulsar como intermediador das mensagens. Nesse ambiente, o cliente envia requisições ao servidor, que executa operações como resposta textual, alteração de arquivo e cálculo de função, retornando os resultados por meio de tópicos de resposta.

Dessa forma, o relatório busca relacionar os conceitos estudados com uma aplicação prática, evidenciando como o Apache Pulsar pode ser utilizado para estruturar a comunicação entre processos distribuídos. Ao longo do documento, são apresentados o referencial teórico, a modelagem da solução, o fluxo de comunicação, as evidências de execução do protótipo e uma avaliação crítica sobre as vantagens, limitações e cenários de uso da tecnologia.

2 Referencial Teórico

2.1 Paradigma de comunicação

O Apache Pulsar adota o paradigma de *Mensageria e Streaming* orientado a eventos, utilizando o modelo de comunicação *Publish-Subscribe (Pub/Sub)*. Nesse paradigma, a troca de mensagens ocorre de forma indireta por meio de tópicos gerenciados pelo middleware, eliminando a necessidade de comunicação direta entre os processos distribuídos.

No modelo *Pub/Sub*, produtores (*publishers*) publicam eventos em tópicos específicos, enquanto consumidores (*subscribers*) assinam esses tópicos para receber as mensagens de interesse. Essa abordagem promove desacoplamento entre os componentes do sistema, permitindo que produtores e consumidores evoluam independentemente, sem conhecimento explícito sobre localização, identidade ou estado uns dos outros.

O Apache Pulsar atua como um *Middleware Orientado a Mensagens* (*Message-Oriented Middleware - MOM*), responsável por abstrair a complexidade da comunicação distribuída. O middleware gerencia o roteamento, armazenamento, persistência e entrega confiável das mensagens, garantindo comunicação assíncrona entre aplicações heterogêneas.

Esse paradigma introduz duas formas principais de desacoplamento:

- **desacoplamento no espaço**: produtores e consumidores não precisam conhecer endereços de rede, portas ou identificadores específicos entre si;
- **desacoplamento no tempo**: emissor e receptor não precisam estar ativos simultaneamente, pois as mensagens podem permanecer persistidas até serem consumidas.

Além disso, o paradigma *Pub/Sub* favorece escalabilidade horizontal, tolerância a falhas e distribuição eficiente de eventos para múltiplos consumidores simultaneamente. Essas características tornam o modelo amplamente utilizado em arquiteturas de microsserviços, sistemas orientados a eventos, aplicações IoT e plataformas financeiras de alta disponibilidade.

2.2 Problema que o middleware resolve

Em sistemas distribuídos, a comunicação direta entre serviços pode gerar forte acoplamento entre os componentes da aplicação. Nesse modelo, produtores e consumidores dependem explicitamente uns dos outros para estabelecer conexão, trocar informações e manter disponibilidade simultânea. Como consequência, falhas de rede, indisponibilidade de serviços ou aumento de carga podem impactar diretamente a estabilidade e a escalabilidade do sistema.

Além disso, arquiteturas baseadas em comunicação síncrona apresentam dificuldades relacionadas à tolerância a falhas, balanceamento de carga e evolução independente dos serviços. Em cenários de alta concorrência, múltiplos processos competindo por recursos podem causar degradação de desempenho e aumento da latência na comunicação.

O Apache Pulsar resolve esses problemas atuando como uma camada intermediária de mensageria distribuída. Em vez de estabelecer comunicação direta, os processos passam a interagir através de tópicos gerenciados pelo broker do middleware. Dessa forma, produtores apenas publicam mensagens, enquanto consumidores as recebem de maneira assíncrona por meio de assinaturas.

© UNIFEI-IMC - 2026 - 1º Semestre

Entre os principais problemas solucionados pelo middleware, destacam-se:

- roteamento e distribuição eficiente de mensagens entre múltiplos consumidores;
- persistência e entrega confiável mesmo em cenários de falha;
- desacoplamento entre produtores e consumidores;
- escalabilidade horizontal por meio da distribuição de carga entre instâncias;
- suporte à comunicação assíncrona e processamento concorrente;
- possibilidade de integração entre aplicações heterogêneas.

No Apache Pulsar, essas funcionalidades são reforçadas pela separação entre computação e armazenamento, permitindo que brokers e mecanismos de persistência sejam escalados independentemente. Essa arquitetura favorece aplicações distribuídas que demandam alta disponibilidade, baixa latência e processamento contínuo de eventos em tempo real.

2.3 Histórico e contexto de uso

O Pulsar foi desenvolvido inicialmente no Yahoo e posteriormente doado à Apache Software Foundation. A plataforma é usada em cenários com alto volume de eventos e baixa latência, como integração de microsserviços, processamento de eventos em tempo real, IoT e plataformas financeiras Apache Pulsar (2026b).

2.4 Exemplos de aplicação

No contexto estudado, destacam-se cenários de:

- divulgação automática de oportunidades de investimento em tempo real;
- integração de aplicações empresariais com sistemas legados;
- monitoramento de dispositivos IoT e processamento de telemetria.

2.5 Arquitetura da tecnologia

No nível conceitual, a arquitetura do Pulsar separa computação e armazenamento:

- **Serving Layer**: brokers stateless para conexão, roteamento e entrega;
- **Storage Layer**: Apache BookKeeper para persistência em ledgers;
- **Metadata Store**: coordenação e metadados.

Essa separação favorece escalabilidade independente entre processamento e armazenamento, característica importante para sistemas distribuídos modernos Apache Pulsar (2026a).

3 Desenvolvimento

3.1 Modelagem da solução

O protótipo foi implementado em Python, com orquestração via Docker Compose, e segue uma arquitetura cliente-servidor com middleware de mensageria.

Elemento	Papel no sistema
`pulsar`	Broker de mensagens responsável por rotear requests e responses.
`server`	Processo que consome requisições, executa operações e publica respostas.
`client`	Processo que envia requisições e aguarda resposta por tópico de retorno.
`PulsarRpcClient`	Classe cliente para fluxo request/reply.
`Request` e `Response`	Contratos de mensagem serializados em JSON.

Tabela 1: Serviços e componentes principais do protótipo.

Item	Identificação
Tópico de request fixo	`prototype-requests`
Tópico de response dinâmico	`replies-{uuid}`
Subscription do servidor	`prototype-server` (Shared)
Subscription do cliente	`client-{uuid}`

Tabela 2: Tópicos e assinaturas do PrototipoSeminario.

3.2 Fluxo de comunicação

1. O cliente cria producer para requests e subscription de resposta.
2. O cliente envia `Request` com `request_id`, operação, payload e `reply_to`.
3. O servidor consome a mensagem no tópico de request.
4. O servidor executa `echo`, `write_file` ou `calculate`.
5. O servidor publica `Response` no tópico de retorno informado.
6. O cliente consome e valida a resposta pelo `request_id`.

3.3 Bibliotecas, parâmetros e comandos

Bibliotecas e ferramentas: Python 3.10, `pulsar-client`, Docker e Docker Compose PyPI (2026).

Parâmetros principais:

- `PULSAR_URL`: endereço do broker;
- `REQUEST_TOPIC`: tópico de requisições;
- `SERVER_FILE_DIR`: diretório de escrita no servidor.

3.4 Diagramas

Os diagramas foram exportados para PNG e inseridos diretamente no relatório.

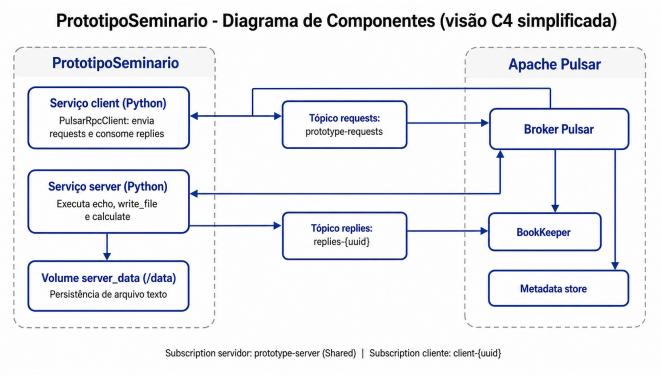

Figura 1: Diagrama de componentes do PrototipoSeminario.

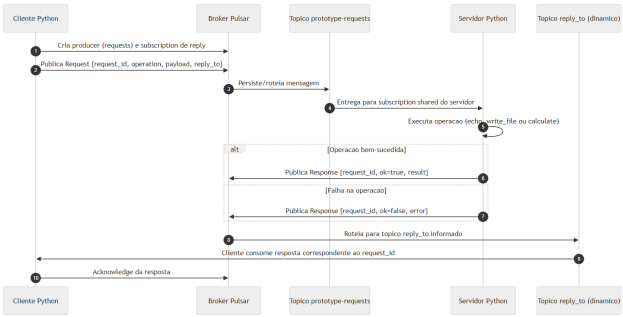

Figura 2: Diagrama de sequência do fluxo request/reply.

4 Resultados e Avaliação

4.1 Validação funcional do protótipo

Foram executados testes fim a fim com Apache Pulsar em funcionamento, validando as três operações obrigatórias e um cenário de erro controlado.

ID	Objetivo do teste	Status
T1	Validar resposta de texto (`echo`)	PASS
T2	Validar alteração de arquivo (`write_file`)	PASS
T3	Validar cálculo de função (`calculate`)	PASS
T4	Validar tratamento de erro para função inválida	PASS

Tabela 3: Resumo dos testes executados.

4.2 Evidências de execução

A Tabela 4 apresenta os principais registros coletados durante a execução do protótipo.

Evidência	Resultado observado
Retorno do cliente para o teste `echo`	`ok=true` e mensagem recebida do servidor, confirmando a conectividade entre cliente, broker e servidor.
Retorno do cliente para o teste `calculate`	`ok=true`, função `multiply` executada corretamente e resultado numérico 42.
Log do servidor	Registro de requisições contendo `request_id` e `reply_to` para as operações `echo`, `write_file` e `calculate`.
Persistência no servidor	Arquivo `registro_e2e.txt` criado e atualizado no diretório de dados do servidor.

Tabela 4: Evidências técnicas de execução.

4.3 Avaliação crítica

- **Vantagens:** o uso do Apache Pulsar permitiu desacoplamento entre os processos, persistência nativa de mensagens e possibilidade de escalabilidade horizontal.
- **Limitações:** a solução apresenta maior complexidade operacional quando comparada à comunicação direta entre processos, além de exigir monitoramento do broker em um ambiente real.
- **Cenários adequados:** o middleware é indicado para integração assíncrona entre serviços, processamento de eventos e distribuição de mensagens para múltiplos consumidores.
- **Cenários menos indicados:** a abordagem pode ser excessiva para sistemas pequenos, com baixa concorrência e comunicação predominantemente síncrona.

4.4 Conclusão

O protótipo atendeu aos requisitos obrigatórios da disciplina para comunicação distribuída com middleware. As operações implementadas foram validadas por meio de testes funcionais, demonstrando a troca de mensagens entre cliente e servidor, a execução de funções remotas, o tratamento de erro controlado e a persistência de dados no servidor.

Dessa forma, os resultados obtidos indicam que o Apache Pulsar foi aplicado de maneira adequada ao problema proposto, servindo como camada intermediária de comunicação e contribuindo para o desacoplamento entre os componentes do sistema distribuído.

4.5 Atribuições Individuais

A seguir, detalham-se os papéis e as responsabilidades primárias de cada integrante:

- **João Leão**
  Papel principal: Organização e estruturação.
  Responsabilidades específicas: introdução da apresentação, coordenação geral do projeto, estruturação do roteiro e conclusão do seminário.
- **Rodrigo Braz**
  Papel principal: Problema, Paradigma e Modelagem.
  Responsabilidades específicas: análise do problema que o Apache Pulsar resolve, explicação do paradigma de comunicação *Publish-Subscribe* (Pub/Sub) e modelagem da solução proposta.
- **Breno Vieira**
  Papel principal: Arquitetura e Fluxo.
  Responsabilidades específicas: detalhamento da arquitetura multicamadas do Apache Pulsar e descrição do fluxo de comunicação, incluindo diagramas de componentes e de sequência.
- **Pedro Venâncio**
  Papel principal: Implementação.
  Responsabilidades específicas: desenvolvimento do protótipo cliente/servidor, codificação das operações e demonstração prática da solução.
- **Wesley Batista**
  Papel principal: Testes, Validação e Análise Crítica.
  Responsabilidades específicas: execução e validação dos testes funcionais, análise crítica das vantagens e limitações do Apache Pulsar e identificação de cenários de uso.

4.6 link Projeto do GitHub

https://github.com/pdrVenancio/PrototipoSeminario

Referências

Apache Pulsar (2026a). Python client library. https://pulsar.apache.org/docs/client-libraries/python/. Acesso em: 04 maio 2026.

Apache Pulsar (2026b). Run pulsar in standalone mode with docker. https://pulsar.apache.org/docs/en/standalone-docker/. Acesso em: 04 maio 2026.

PyPI (2026). pulsar-client. https://pypi.org/project/pulsar-client/. Acesso em: 04 maio 2026.